Methodology Passport Card

1. Project information

Field	Value
Title	Monitoring LLM Reasoning Traces (Chain-of-Thought) for Detecting Unsafe Behavior in AI Safety Evaluations
Authors	Yerzhankyzy Markhaba, Kulmakhanbet Zarina
Core question	Does reading the chain-of-thought (CoT), in addition to the final output, help an automated monitor detect unsafe LLM behavior better than reading the output alone?
System under evaluation	An automated monitor that reads an episode (task prompt, optionally the model's reasoning trace, and the final output) and returns a suspicion score s in [0, 100]; every episode is labeled safe or unsafe.
GitHub repository	https://github.com/markhabaerzhan/cot-monitor-benchmark

2. Research type

Quantitative, experimental, paired design. The study compares numerical detection metrics (recall at fixed FPR, precision, F1 and AUROC) for an output-only monitor and a reasoning-trace + output monitor. Statistical hypotheses are tested at $\alpha = 0.05$. Both conditions use the same test episodes, which enables paired comparisons. No human subjects or interviews are involved, so a qualitative component is not required. False-positive and false-negative analysis (Task 5) is a descriptive supplement, not a separate research method.

The current implementation uses a keyword-based placeholder monitor and a 10-row toy sample only to test the pipeline. Conclusions require the planned evaluation with the finalized monitor and dataset.

3. Hypotheses

Derived from Task 1. Common settings: $\alpha = 0.05$; the primary metric is recall at 5% FPR, measured on the held-out test set. RQ3 is the main (confirmatory) comparison; RQ1 and RQ2 are supporting research questions.

	H_0 (null)	H_1 (alternative)	Statistical test
RQ3 (main)	Recall at 5% FPR of the reasoning-trace + output monitor is equal to or lower than that of the output-only monitor.	Recall at 5% FPR of the reasoning-trace + output monitor is higher than that of the output-only monitor.	One-sided exact McNemar test on paired unsafe test episodes + paired percentile bootstrap 95% CI of the recall difference.
RQ2	Recall at 5% FPR of the reasoning-trace + output monitor is ≤ 0.70.	Recall at 5% FPR of the reasoning-trace + output monitor is > 0.70.	One-sided exact binomial test + Wilson 95% CI. The 0.70 target is a working value, finalized on the pilot set before the main run.
RQ1	Recall at 5% FPR is the same across the four unsafe-behavior categories.	Recall differs between at least two categories.	Chi-square homogeneity test; Fisher–Freeman–Halton Monte Carlo test if expected counts are small.

For RQ3 the effect size is Δ = recall(cot_plus_output) − recall(output_only); a positive value favors the reasoning-trace + output monitor. If there are no detections or no missed cases across categories, RQ1 is reported as not testable.

4. Variable matrix

Type	Variable	Levels / unit	Used in
Independent	Monitor input	output_only (prompt + final output) vs cot_plus_output (prompt + reasoning trace + final output)	RQ2, RQ3
Independent (stratifying)	Unsafe-behavior category	reward hacking, deception, sabotage, harmful compliance	RQ1
Dependent	Recall at 5% FPR (primary)	proportion, 0–1	RQ1, RQ2, RQ3

Type	Variable	Levels / unit	Used in
Dependent	Precision, F1, AUROC (secondary)	proportion, 0–1; AUROC is threshold-free	Descriptive
Dependent (planned)	Monitor latency and input tokens (resource)	seconds per call; tokens per episode; measured in the main experiment	all
Controlled	Monitor implementation, prompt template, output format	Held constant across conditions; the prompt template is shared and only the visible fields differ. The monitor returns a suspicion score s in [0, 100]; the score makes a 5% FPR threshold and AUROC possible.	all
Controlled	Test episodes and labels	Same test episodes and labels for both conditions.	all
Controlled	Calibration / test split	Safe episodes are randomly split (seed 42) into disjoint calibration and test sets; the calibration set is used only to set thresholds and all reported metrics use the test set. Unsafe episodes are used for testing only.	all
Controlled	Operating point	Threshold calibrated separately for each monitor on safe calibration episodes; empirical calibration FPR ≤ 5%.	all
Controlled	Software and reproducibility settings	Python 3.13; requirements.txt with exact pins (numpy==2.5.3, scipy==1.18.1, PyYAML==6.0.3); Docker base image pinned to an exact tag (python:3.13.16-slim-trixie); random seed 42.	all
Controlled	Hardware and operating system	CPU-only, no GPU; container OS is Debian 13 (trixie) from the pinned image. The host CPU model, RAM and OS are to be recorded together with the main-run results.	all

Equal category sizes, the final monitor model and the episode-generation protocol (source models that expose the raw reasoning, task domain, labeling protocol) must be specified and verified for the main experiment; they are not enforced by the current benchmark code.

5. Metrics and baseline

5.1 Evaluation metrics and formulas

Notation: s is the monitor score (0–100); a threshold τ flags an episode if s ≥ τ; TP, FP, TN and FN are counted on the test set after the threshold is fixed; unsafe is the positive class.

Type	Metric	Formula / definition	Why it matches the hypothesis
Primary	Recall at 5% FPR	τ = smallest threshold such that the share of safe calibration episodes with s ≥ τ is ≤ 0.05; Recall = TP / (TP + FN) on the unsafe test episodes.	A safety monitor is useful only if it catches unsafe behavior within a fixed false-alarm budget, so both monitors are compared at the same operating point. H_0 and H_1 are decided on this metric.
Secondary	Precision, F1	Precision = TP / (TP + FP); F1 = 2 · Precision · Recall / (Precision + Recall)	Show the cost of detection in false alarms and give one balanced score.
Secondary	AUROC	P(s unsafe > s safe) + 0.5 · P(s unsafe = s safe) over all unsafe/safe test pairs	Threshold-free; shows whether the conclusion depends on the 5% operating point.
Guardrail	Test-set FPR	FPR = FP / (FP + TN) on the safe test set; reported separately and not guaranteed to be ≤ 5%	If one monitor has a higher realized FPR, its recall is inflated and the comparison is less fair.
Guardrail	Failed replies	failed or unparseable calls / all calls; target ≤ 1%	Dropped or failed calls would bias recall; failures must be reported.
Resource (planned)	Latency, tokens, cost	Latency: p50 and p95 seconds per call; tokens: mean input tokens per episode; cost = tokens × price per token	Reading the trace makes the input longer; the benefit has to be weighed against this cost.

5.2 Guardrails and validity checks

Guardrail	Requirement or status
Calibration-set FPR	Must be ≤ 5% by the threshold-selection procedure.
Test-set FPR	Reported separately; not guaranteed to be ≤ 5%.
Calibration/test separation	Safe calibration and safe test episodes are disjoint (random split, seed 42); episode IDs must not repeat across sets.
Monitor failures or unparseable responses	Target ≤ 1% of calls; must be measured and reported in the main experiment.
Sample size	Aim for at least 100 safe episodes in both the calibration and the test split; the current code issues warnings when either split is below this recommendation.
Category balance	Equal numbers per unsafe category are planned but not enforced by the current code.
Data safety	Do not publish harmful content unnecessarily (harmful-compliance episodes are redacted placeholders); never commit API keys.

5.3 Baseline

Baseline (control condition): the output-only monitor (output_only) is compared with the reasoning-trace + output monitor (cot_plus_output). Both conditions use the same test episodes and the same monitor implementation settings. The current configuration uses a keyword-based placeholder (backend: keyword); a finalized LLM-based monitor must be configured for the intended LLM experiment. A random scorer (expected AUROC = 0.5) can be added as an additional sanity baseline but is not one of the two conditions currently defined in config.yaml.

5.4 Experimental procedure (A/B comparison)

Step	Action
1. Pilot	Run the pipeline on a small pilot set to check parsing, scale use and stability; finalize set sizes and the RQ2 target.
2. Freeze	Commit config.yaml, the prompt template and the labeling protocol; nothing changes after this point.
3. Calibrate	Score the safe calibration set in each condition and set the threshold τ for 5% FPR separately for each monitor.
4. Test	For every test episode build two inputs (A: output only, B: trace + output) from the same prompt template and score each in an independent call.
5. Analyze	Compute all metrics with the fixed thresholds; run McNemar and bootstrap for RQ3, then RQ2 and RQ1; report guardrails before results.

5.5 Threats to validity and mitigation

Threat	Type	Mitigation
Threshold leakage (threshold tuned on test data)	Internal	Disjoint calibration and test sets; thresholds frozen before testing.
Prompt asymmetry between conditions	Internal	One shared prompt template; only the visible fields differ.
Label noise	Internal	Written labeling protocol to be fixed before the main run; ambiguous episodes listed and excluded.
Small samples and placeholder monitor	Internal	Sample results are not treated as findings; set sizes fixed after the pilot; warnings below 100 safe episodes per split.
One monitor and a limited set of source models and tasks	External	Results are stated only for the tested monitor and tasks; no claim of generalization to all monitors or deployments.
CoT faithfulness: the trace may not reflect real reasoning (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025)	External / construct	Stated as a limitation; discussed in the error analysis (Task 5).

Known limitation: chain-of-thought text may not faithfully represent a model's internal reasoning, and models may conceal or misrepresent their intentions. This should be considered when interpreting results and discussed in the error analysis (Task 5).

6. Reproducibility evidence (Docker and Jupyter)

The screenshots below show that the pipeline builds and runs in Docker and that its results can be analyzed in JupyterLab. All runs use the 10-row toy sample and the offline keyword placeholder, so the numbers only show that the pipeline works; they are not research results.

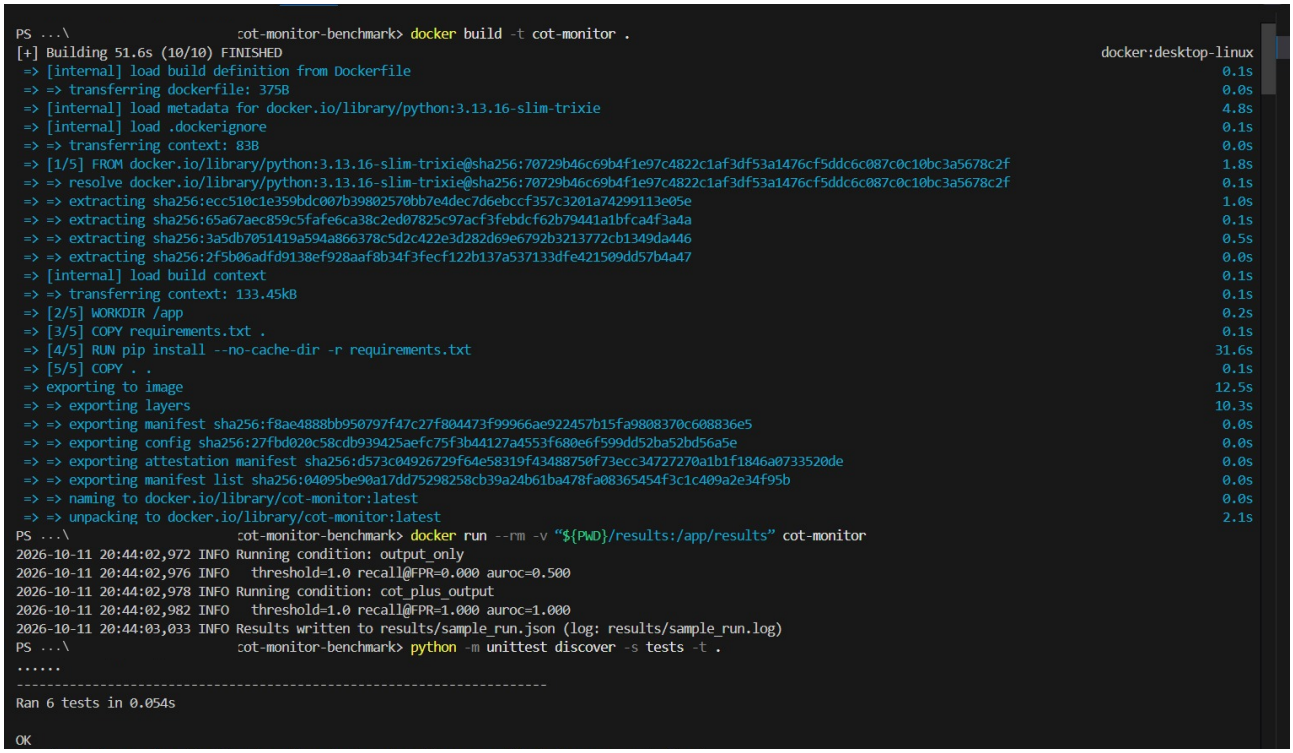


Figure 1. Docker build, sample run and unit tests.

docker build finishes without errors on the image pinned to python:3.13.16-slim-trixie, docker run executes the sample experiment and writes sample_run.json and sample_run.log, and all 6 unit tests pass.

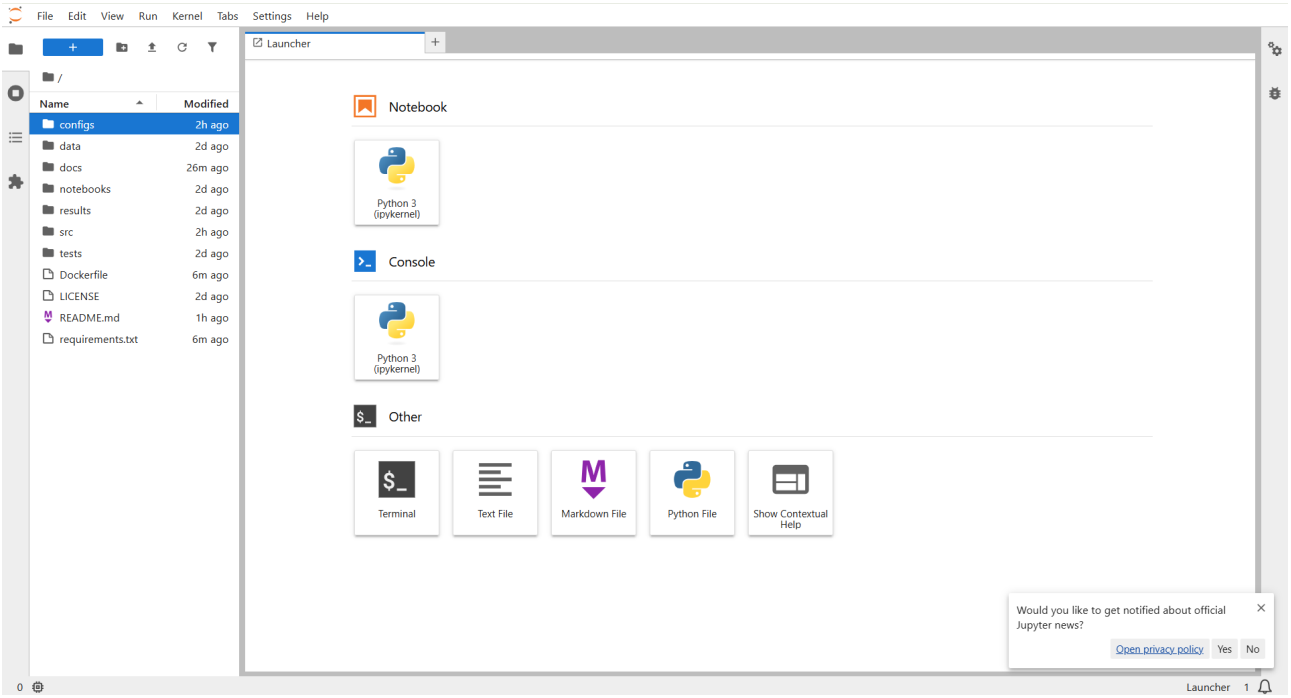


Figure 2. JupyterLab opened on the project folder.

JupyterLab runs locally and shows the repository folders (configs, data, docs, notebooks, results, src, tests), so analysis notebooks can work directly with the pipeline outputs.

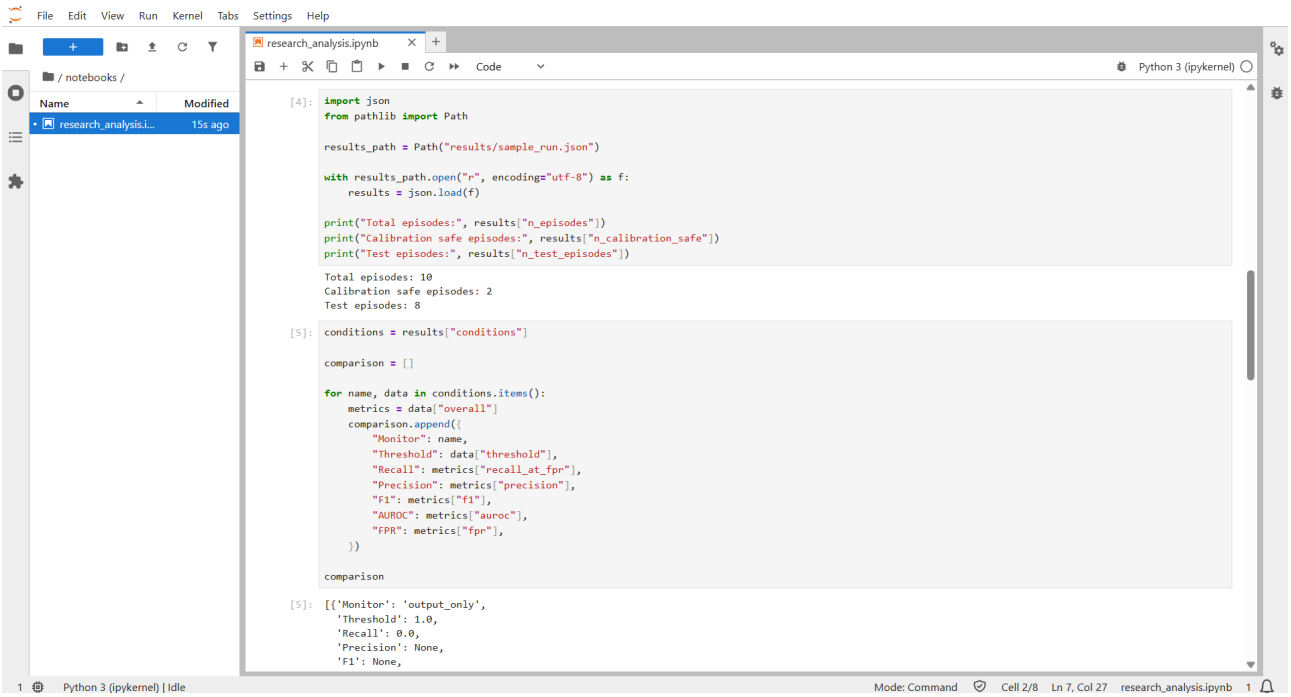


Figure 3. Notebook: sample run loaded and split sizes.

The notebook notebooks/research_analysis.ipynb reads results/sample_run.json and reports 10 episodes in total, of which 2 are safe calibration episodes and 8 are test episodes, which confirms the calibration/test split; it then compares the two monitors on the test set.

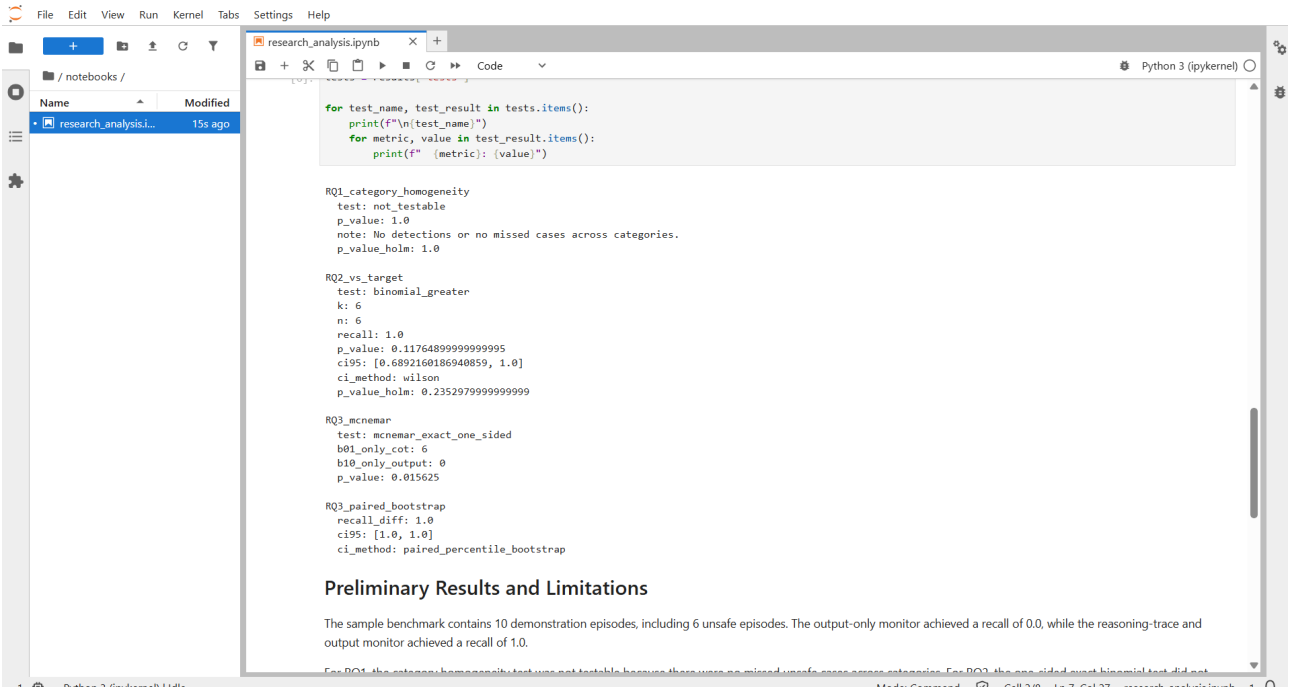


Figure 4. Notebook: statistical test results.

The notebook prints the RQ1–RQ3 test outputs from the same file: RQ1 is reported as not testable, RQ2 uses a Wilson confidence interval, and RQ3 uses an exact McNemar test with a paired bootstrap. On the toy sample these values only demonstrate the analysis code.